

Let's create a simple but realistic strategy implementation:

```
1. def backtest_strategy(data, initial_capital=100000.0, position_size=0.95,
2.                        commission_per_share=0.005, slippage_percent=0.05):
3.     """
4.     Backtest a trading strategy with realistic assumptions
5.
6.     Parameters:
7.     data (pd.DataFrame): DataFrame with price data and signals
8.     initial_capital (float): Starting capital
9.     position_size (float): Percentage of capital to allocate per trade (0-1)
10.    commission_per_share (float): Commission cost per share traded
11.    slippage_percent (float): Estimated slippage as percentage of price
12.
13.    Returns:
14.    pd.DataFrame: DataFrame with backtest results
15.    """
16.    # Create a copy of the data
17.    backtest = data.copy()
18.
19.    # Initialize portfolio columns
20.    backtest['Position'] = 0 # Number of shares held
21.    backtest['Cash'] = initial_capital # Cash balance
22.    backtest['Holdings Value'] = 0 # Value of shares held
23.    backtest['Portfolio Value'] = initial_capital # Total portfolio value
24.    backtest['Trade'] = 0 # Share quantity traded (+ for buy, - for sell)
25.    backtest['Trade Cost'] = 0 # Cost of trading (commissions + slippage)
26.
27.    # Process each day in the backtest
28.    for i in range(1, len(backtest)):
29.        # Get yesterday's and today's data
30.        prev = backtest.iloc[i-1]
31.        curr = backtest.iloc[i]
32.
33.        # Default to carrying forward yesterday's position
34.        backtest.loc[backtest.index[i], 'Position'] = prev['Position']
35.        backtest.loc[backtest.index[i], 'Cash'] = prev['Cash']
```